

Module 10

REST APIs & Webhooks

Consuming third-party APIs, OAuth, and outbound webhooks

ContentAutomation2 — Backend Learning Series
Zero Prerequisites → Production-Ready Code

1. What is a REST API?

REST (Representational State Transfer) is a convention for designing APIs. Resources are accessed via URLs. Operations are expressed with HTTP methods: GET (read), POST (create), PUT/PATCH (update), DELETE (remove). Data is exchanged as JSON.

REST API pattern examples:

```
GET /users          → list all users
GET /users/42       → get user with id=42
POST /users         → create a new user (body: {"name": "Alice"})
PUT /users/42       → fully replace user 42
PATCH /users/42    → partially update user 42 (body: {"name": "Alice Smith"})
DELETE /users/42    → delete user 42
```

2. Making HTTP Requests with httpx

httpx is the modern Python HTTP client with both sync and async support. Use it to call external APIs from your backend.

```
pip install httpx
```

```
import httpx
```

```
# Synchronous
```

```
resp = httpx.get("https://api.example.com/users/42")
print(resp.status_code)    # 200
print(resp.json())         # {"id": 42, "name": "Alice"}
```

```
# With headers (auth token)
```

```
resp = httpx.get(
    "https://api.example.com/protected",
    headers={"Authorization": "Bearer YOUR_TOKEN"}
)
```

```
# POST with JSON body
```

```
resp = httpx.post(
    "https://api.example.com/users",
    json={"name": "Bob", "email": "bob@example.com"},
    headers={"Authorization": "Bearer YOUR_TOKEN"},
)
print(resp.json())
```

3. Async HTTP Requests

```
import httpx
import asyncio
```

```
async def fetch_user(user_id: int) -> dict:
    async with httpx.AsyncClient() as client:
        resp = await client.get(
            f"https://api.example.com/users/{user_id}",
            headers={"Authorization": "Bearer TOKEN"},
            timeout=10.0,    # 10 second timeout
        )
```

```

        resp.raise_for_status()    # raises HTTPStatusError for 4xx/5xx
        return resp.json()

async def main():
    # Fetch 10 users concurrently
    results = await asyncio.gather(*[fetch_user(i) for i in range(1, 11)])
    print(results)

asyncio.run(main())

```

4. API Authentication Patterns

API Key: simplest — pass a secret key in a header. Used by most simple APIs.

```

# API key in header
headers = {"X-API-Key": "your-api-key"}

# API key as Bearer token
headers = {"Authorization": "Bearer your-api-key"}

# API key as query parameter (less secure)
resp = httpx.get("https://api.example.com/data?api_key=your-key")

```

OAuth 2.0: used by LinkedIn, Twitter, Google. More complex but more secure.

```

# OAuth 2.0 flow (simplified):
# Step 1: Direct user to provider's auth URL
auth_url = "https://linkedin.com/oauth/v2/authorization?client_id=...&scope=w_member_social"

# Step 2: User logs in, provider redirects back to your app with a code
code = "abc123"    # from query string

# Step 3: Exchange code for an access token
resp = httpx.post("https://linkedin.com/oauth/v2/accessToken", data={
    "grant_type": "authorization_code",
    "code": code,
    "client_id": "YOUR_CLIENT_ID",
    "client_secret": "YOUR_SECRET",
    "redirect_uri": "https://yourapp.com/callback",
})
access_token = resp.json()["access_token"]

# Step 4: Use access_token to make API calls
resp = httpx.post(
    "https://api.linkedin.com/v2/ugcPosts",
    headers={"Authorization": f"Bearer {access_token}"},
    json={...},
)

```

5. Twitter/X via Tweepy

```

pip install tweepy

import tweepy
# OAuth 1.1a (for posting as a user account)

```

```

client = tweepy.Client(
    consumer_key="...",
    consumer_secret="...",
    access_token="...",
    access_token_secret="...",
)

# Post a tweet
response = client.create_tweet(text="This is an automated tweet! #fintech")
tweet_id = response.data["id"]
print(f"Posted tweet: {tweet_id}")

# Get tweet metrics
tweet = client.get_tweet(
    tweet_id,
    tweet_fields=["public_metrics"]
)
metrics = tweet.data.public_metrics
print(f"Likes: {metrics['like_count']}, Retweets: {metrics['retweet_count']}")

```

6. Exponential Backoff — Retry on Failure

```

import asyncio
import httpx

async def post_with_retry(url: str, json_data: dict, max_retries: int = 3) -> dict:
    delay = 1.0 # start with 1 second delay

    for attempt in range(max_retries + 1):
        try:
            async with httpx.AsyncClient() as client:
                resp = await client.post(url, json=json_data)
                resp.raise_for_status()
                return resp.json()

        except httpx.HTTPStatusError as e:
            if e.response.status_code == 429: # rate limited
                print(f"Rate limited. Waiting {delay}s (attempt {attempt+1})")
                await asyncio.sleep(delay)
                delay *= 2 # exponential: 1s, 2s, 4s, 8s...
            elif attempt == max_retries:
                raise
            else:
                await asyncio.sleep(delay)
                delay *= 2

```

7. Webhooks — Receiving Events from External Services

A **webhook** is an HTTP endpoint YOU expose that external services call when something happens. Instead of polling 'did anything change?', the service pushes data to you the moment it does.

```

from fastapi import FastAPI, Request, HTTPException
import hmac, hashlib

```

```

app = FastAPI()

WEBHOOK_SECRET = "your-secret"

@app.post("/webhook/github")
async def github_webhook(request: Request):
    # Verify the signature to prevent fake requests
    signature = request.headers.get("X-Hub-Signature-256", "")
    body = await request.body()
    expected = "sha256=" + hmac.new(
        WEBHOOK_SECRET.encode(), body, hashlib.sha256
    ).hexdigest()

    if not hmac.compare_digest(signature, expected):
        raise HTTPException(status_code=401, detail="Invalid signature")

    payload = await request.json()
    event = request.headers.get("X-GitHub-Event")
    print(f"Got {event} event: {payload.get('action')}")
    return {"status": "received"}

```

8. Outbound Webhooks — Sending to Slack

```

import httpx

def send_slack_alert(webhook_url: str, message: str) -> None:
    """Send a message to a Slack channel via Incoming Webhook."""
    try:
        resp = httpx.post(
            webhook_url,
            json={"text": message},
            timeout=5.0,
        )
        resp.raise_for_status()
    except Exception as exc:
        print(f"Slack alert failed: {exc}")    # non-fatal

# Usage:
send_slack_alert(
    "https://hooks.slack.com/services/YOUR/SLACK/WEBHOOK",
    ":rotating_light: Research agent cost alert: $2.50 this run!"
)

```

Note: ContentAutomation2 sends Slack alerts for cost overruns and site failure events. See [backend/app/utls/slack.py](#).

9. Practice Exercises

- Exercise 1: Use `httpx` to fetch <https://jsonplaceholder.typicode.com/posts> and print the first 3 titles.
- Exercise 2: Make the same call async using `AsyncClient` and `asyncio.gather` for 5 concurrent requests.
- Exercise 3: Add a 5-second timeout and handle the `TimeoutError` gracefully.

- Exercise 4: Implement `post_with_retry` with 3 retries and exponential backoff.
- Exercise 5: Create a FastAPI `/webhook` endpoint that prints any POST body it receives.
- Exercise 6: Set up a Slack Incoming Webhook (free from slack.com/apps) and send a test message.